MZ2POL-01: Introduction - Why Migrate from Management Zones

> **Series:** MZ2POL — Management Zone to Policy Migration | **Notebook:** 2 of 10 | **Created:** December 2025 | **Last Updated:** 08/04/2026

Overview

This notebook introduces the migration from **Management Zones (MZs)** in classic Dynatrace to the modern **Policies, Boundaries, and Segments** framework. Understanding why this migration is necessary and what benefits it brings is essential for planning a successful transition.

> **Note:** This series is for organizations **migrating from existing Management Zones**. If you're setting up IAM governance from scratch (greenfield), see the **IAM** series for comprehensive IAM administration guidance.

Table of Contents

Target Audience

- Dynatrace administrators managing access control
- Platform engineers responsible for multi-tenant configurations
- Security teams overseeing data access policies
- Teams currently using Management Zones for data filtering

Learning Objectives

By the end of this notebook, you will:
1. Understand why Management Zones are being deprecated
2. Know the key differences between MZs and the new model
3. Recognize the benefits of Policies, Boundaries, and Segments
4. Distinguish the three jobs a Management Zone conflates — access control, filtering, and alerting — and see how each fans out to a different construct (Policies + Boundaries, Segments, and problem-triggered workflows, respectively)
5. Identify your current MZ usage patterns for migration planning

Sprint 1.337 (April 2026): Migration Urgency Confirmed by API

Sprint 1.337 of the **Dynatrace API** explicitly deprecated `managementZone` and `serviceTag` properties on the calculated-metrics service-configuration endpoints. This is the strongest signal yet that the Management Zone-based access control model is on the legacy path — the calculated-metrics surface, one of the most-used Configuration API touch points, is now actively pruning MZ references.

What this means for the MZ → Policies migration program:

- **Argument for prioritization:** existing calculated metrics scoped by Management Zone keep working, but new calculated metrics should not be authored against `managementZone`. This forces MZ-dependent automation onto the Policies + Boundaries model sooner rather than later.
- **Audit angle:** as part of the assessment in MZ2POL-03, query for any custom calculated metrics that reference `managementZone` or `serviceTag` — they're now flagged-for-rewrite by the API itself.
- **Tagging story aligns:** sprint 1.337 also added OneAgent primary fields and primary tags at the source (top-level `dt.security_context`, `primary_tags.team`, etc.) on

Latest Dynatrace tenants. These are the Gen3-native carriers of the same signals MZs used to encode — making the policy-and-boundary model more powerful and reducing the residual reasons to keep MZs around.

The migration narrative documented below is unchanged but now has direct API-level support behind it.

---

1. The Evolution of Access Control in Dynatrace

What Are Management Zones?

Management Zones have been the primary mechanism for:
- **Data filtering**: Limiting what data users see in the UI
- **Access control**: Restricting user access to specific entities
- **Multi-tenancy**: Separating data for different teams, regions, or business units

Why the Change?

Management Zones were designed for **classic Dynatrace** and have fundamental limitations:

Limitation	Impact
Precalculated attributes	Performance bottleneck at scale
Not compatible with Grail	Cannot filter Grail-stored data
Limited flexibility	Complex multi-dimensional filtering difficult
Tight coupling	Mixing "what" and "where" in a single construct

The New Platform Architecture

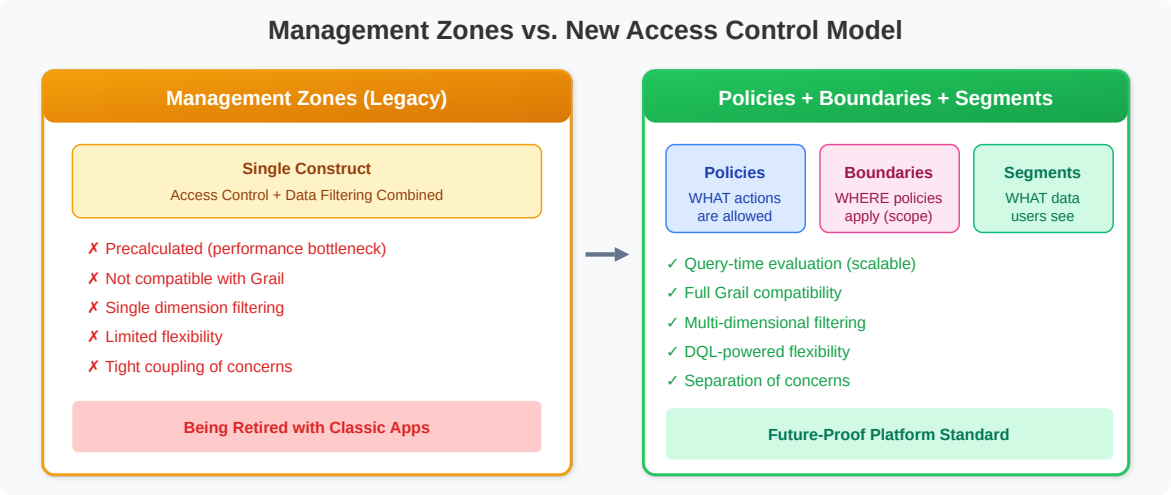
The latest Dynatrace platform uses **Grail** as its data lakehouse. Grail:
- Does **NOT** support Management Zones
- Uses **storage:** fields for record-level access control

- Requires **IAM policies** for data access
- Leverages **Segments** for query-time filtering

---

2. The New Access Control Model

The modern Dynatrace access control framework is based on **ABAC (Attribute-Based Access Control)** and consists of three key components:



Policies

What they do: Define WHAT actions users can perform

- Encapsulate permissions against protected platform resources and data
- Leverage user, resource, data, and contextual attributes
- Can be default (read-only) or custom

Boundaries

What they do: Define WHERE policies apply (scope restrictions)

- Bundle restrictions on record and/or resource level
- Enable re-usability across multiple policy assignments
- Decouple "what" from "where" for easier management
- Used together with policies when assigning to groups

Segments

What they do: Provide dynamic data filtering at query time

- Reusable, pre-defined filter conditions using DQL
- Support variables for dynamic filtering
- Multi-dimensional - can be layered for precise filtering
- Replace MZ filtering for cross-app data segmentation

Comparison: Management Zones vs. New Model

Aspect	Management Zones	Policies + Boundaries + Segments
Scope	Classic apps only	All apps including Grail-based
Performance	Precalculated (bottleneck)	Query-time evaluation (scalable)
Flexibility	Single dimension	Multi-dimensional
Reusability	Limited	High (components are decoupled)
Data filtering	Built into MZ	Segments (separate concern)
Access control	Built into MZ	Policies + Boundaries (separate)

---

3. Understanding Your Current Management Zone Usage

Before migrating, audit your current MZ implementation.

> ⚠️ **Important:** Management Zone configurations **cannot be queried via DQL**.
>
> **Options for viewing MZ configurations:**
> 1. **Dynatrace UI (Recommended):** Settings → Management Zones
> 2. **SDK Analysis:** See **MZ2POL-00: SDK Management Zone Analysis Tool** for comprehensive analysis including rule patterns, coverage metrics, and migration readiness.

Check Entity Distribution by Management Zone

You can query entity distribution across MZs using DQL:

```
// Count entities per Management Zone
// Helps understand data distribution for Segment planning
fetch dt.entity.service
| expand mz = managementZones
| summarize serviceCount = count(), by:{managementZone = mz}
| sort serviceCount desc
| limit 20

// Note: dt.entity.* is deprecated, but this is a migration-ASSESSMENT query — it inspects
// the classic constructs this migration replaces, so keep the classic query above:
//   - management zones have NO Smartscape node equivalent; they are what this migration
//     replaces with segments and policies.
```

---

4. Benefits of Migration

Scalability

- **Query-time filtering** instead of precalculated attributes
- Handle orders of magnitude higher data volumes
- No performance degradation with complex filtering

Flexibility

- **Multi-dimensional segments** - layer multiple filters
- **Dynamic variables** - adapt to user context
- **DQL-powered** - full query language for conditions

Separation of Concerns

- **Policies**: What can be done (permissions)
- **Boundaries**: Where it can be done (scope)

- **Segments**: What data to show (filtering)

Future-Proof

- Management Zones will be **retired** with classic apps
- All new Dynatrace apps use Grail and Segments
- Alerting profiles give way to **problem-triggered workflows** — not to Segments (see §5)

> **Segments do not scope alerting.** A common misreading of this migration is that alerting profiles will eventually accept a segment the way a dashboard does. They will not. Alerting profiles are scoped by Management Zone plus severity-rule tag matching; their Gen3 successor is a problem-triggered workflow whose trigger filters on affected-entity tags and an optional DQL matcher. Segments touch alerting in exactly one place — scoping an *anomaly detector* — and never scope triggers, notifications, or problem visibility. §5 gives the full mapping.

---

5. Migration Timeline Considerations

Current State (2025)

- Management Zones **still work** for classic apps
- New apps (Dashboards, Services app, etc.) require Segments
- Hybrid approach possible during transition

What to Migrate

Management Zones do several unrelated jobs at once, and those jobs migrate to **different** places. Read this table as a fan-out, not a rename — there is no single construct that "replaces" a Management Zone.

Current MZ Use Case	New Solution
User access restriction	Policies + Boundaries

Current MZ Use Case	New Solution
Data filtering in UI	Segments
Alerting profile scoping	**Problem-triggered workflows** — trigger filters on affected-entity tags + optional DQL matcher
Problem *visibility* (who may see which problems)	IAM record-level policy on `dt.security_context`
Anomaly detector scoping	Segments (the one place segments touch alerting)
Dashboard filtering	Segments
API access control	Policies + Boundaries

The three axes, kept separate

The single most common planning error is collapsing these into one:

Axis	Question it answers	Mechanism
Routing	Who gets paged?	Workflow trigger: affected-entity tags + DQL matcher
Visibility	Who may see the problem at all?	IAM policy on `dt.security_context`
Filtering	What does a user see in the UI?	Segments

Segments are not an access-control mechanism and are not a routing mechanism. Anyone holding `storage:filter-segments:read` can apply any segment.

> ⚠️ **The Management Zone filter has no successor inside the alerting model.** Dynatrace's upgrade guide states it plainly: *"Management Zone filter — No longer supported. Use Grail record-based field filters instead."* This means the routing dimension must be carried by something the workflow trigger can actually match on — entity **tags**, or `dt.security_context`. If your MZ rules are computed from name patterns or entity selectors rather than tags, that enrichment work is a **prerequisite**, not a follow-up. See MZ2POL-05 §4 for the rule-to-dimension classification.
>
> ⚠️ **Duration-based suppression has a successor — verify it before assuming otherwise.** An alerting profile can delay notification until a problem has been open

> longer than *N* minutes (`delayInMinutes`). The problem trigger's **Delay** option postpones *"the trigger until the problem has been open for at least the configured duration"* — 5, 10, 15, 30, 60, 120, 240, 1440, or 10080 minutes, evaluated on `dt.duration_marker` . Note that the alert-notification upgrade guide still states there is *"currently no alternative"* for this — the two pages conflict, and the upgrade guide appears not to have been re-tensed. **Confirm the Delay option in your own tenant**, then inventory `delayInMinutes` across every profile and map each value onto a Delay setting. See MZ2POL-09 §6.1.

Migration Phases

1. **Assessment**: Audit current MZ usage (this notebook)
2. **Planning**: Map MZs to Policies/Boundaries/Segments
3. **Implementation**: Create new access control constructs
4. **Validation**: Test access and filtering
5. **Cutover**: Transition users to new model
6. **Cleanup**: Remove deprecated MZ configurations

Summary

In this notebook, you learned:

1. **Why migrate**: Management Zones don't work with Grail and will be retired
2. **The new model**: Policies (what) + Boundaries (where) + Segments (filter)
3. **Key benefits**: Scalability, flexibility, separation of concerns
4. **The fan-out**: one MZ's jobs migrate to *different* constructs — access to policies/boundaries, UI filtering to segments, alert routing to workflow triggers, problem visibility to `dt.security_context`
5. **Assessment queries**: How to audit your current MZ usage

Next Steps

Continue to **MZ2POL-02: Understanding the New Access Control Model** to dive deeper into:
- Policy structure and syntax
- Boundary configuration
- Segment creation and management

For the alerting side of the migration specifically, see **WFLOW-02** (problem-trigger configuration) and **WFLOW-04** (tag- and ownership-based routing).

Additional Resources

- Access Management Concepts
- Policy Boundaries
- Upgrade from RBAC to ABAC
- Segments Documentation
- Upgrade guide — alerting and notifications (DT docs) — the authoritative old→new mapping, including the "Management Zone filter: no longer supported" line and the connector equivalence table
- Alerting profiles (DT docs) — Dynatrace Classic surface; carries the "we recommend using simple workflows" steer
- Use segments in anomaly detection (DT docs) — the one documented intersection of segments and alerting

---